Integration of the FINS Protocol into the Open-Source HMI System FUXA for Omron PLCs

Denny Chrisnanda
Computer Science Department
Binus Online Learning
Bina Nusantara University
denny.chrisnanda@binus.ac.id

Frans Sebastian
Computer Science Department
Binus Online Learning
Bina Nusantara University
frans.sebastian@binus.ac.id

Gilang Hansagita
Computer Science Department
Binus Online Learning
Bina Nusantara University
gilang.hansagita@binus.ac.id

Abstract

Omron PLCs dominate automation at PT XYZ and most of them use FINS protocol. The problem is that almost all open-source HMI platforms—including the lightweight and fully web-based FUXA—do not support FINS natively. To address this issue, this study develops and integrates a dedicated FINS communication module into FUXA. The study followed an iterative approach: literature review, FINS protocol analysis, module implementation, and testing on an Omron CJ2M PLC. The tests showed that the new module can reliably read and write data to/from the PLC with little delay, and the screens in FUXA now show exactly what is really happening on the factory floor. The result is a completely free, fully customizable, browser-based HMI that speaks directly to Omron PLCs without any extra gateway or middleware. A key limitation identified in this study is the alarm detection delay of approximately 1000 ms. The reason is the system has a minimum polling interval for alarms set at 1 second (or 1000 ms). This limits how often it can check and trigger events without changes to the core structure. The study does not address this limitation because it primarily focused on adding the FINS protocol to FUXA. This was meant to enable smooth communication with Omron PLCs over FINS Protocol instead of improving or optimizing existing features like the alarm polling system. This development makes modern open-source tools viable in Omron-dominated plants and helps companies move faster with their digital-transformation projects.

Keywords*—FINS protocol, FUXA, human-machine interface, industrial automation, Omron PLC*

1. Introduction

Programmable Logic Controllers (PLCs) play a crucial role in modern industrial automation systems because they are reliable, respond in real time, and are easy to program [1]. At PT. XYZ, a global automotive parts manufacturer, Omron PLCs are prevalent in the production environment. Internal records show that more than 90% of controlled machines use Omron devices. This dominance means that all higher-layer components in the automation stack must fully support the communication protocols that are part of this system.

Operators can monitor processes, modify parameters, and receive timely alerts when abnormalities occur thanks to Human–Machine Interfaces (HMIs) [2]. Commercial HMI/SCADA systems provided centralized visibility over widespread assets, collecting real-time data from field instruments and allowing operators to issue supervisory commands from control centers.[3], but they usually come with high licensing costs, little customization options, and a high risk of vendor lock-in [4, 5]. As a result, a growing number of manufacturing companies are investigating open-source alternatives that provide unlimited modification potential and no upfront licensing fees [6].

FUXA is an open-source HMI platform introduced in 2019 that has recently gained growing adoption in industry and academic research. It enables the rapid development of web-based HMIs and SCADA systems without requiring web programming expertise, offering native support for major industrial protocols such as OPC UA, Modbus, BACnet, Ethernet/IP, Siemens S7, MQTT, ADS, and MC Protocol/SLMP. Interfaces are created through a visual drag-and-drop editor with configurable widgets, alarms, access control, and historical data logging, and can be deployed on any modern web browser across PCs, panels, and mobile devices.[7, 8, 9]. Its modest resource requirements allow it to run smoothly on single-board computers such as the Raspberry Pi or in cloud environments, making it an attractive choice for remote monitoring applications and broader Industry 4.0 deployments. Despite these strengths, FUXA does not yet include built-in support for Omron's Factory Interface Network Service (FINS)

protocol. FINS, which operates over both UDP and TCP, is a proprietary yet widely deployed protocol that has been adopted across numerous Omron product families and remains extensively used in measurement and control systems worldwide. It continues to serve as the de facto communication standard in facilities dominated by Omron PLCs, primarily due to its highly flexible memory-area addressing scheme that enables fast, low-latency data exchange between controllers and supervisory systems [10]. The absence of native FINS connectivity has therefore remained the principal obstacle preventing many Omron-centric plants—including PT. XYZ—from fully adopting FUXA as a viable, vendor-independent HMI solution. This study directly addresses that limitation by developing and integrating a dedicated FINS communication driver into the FUXA platform.

This study directly addresses this gap by investigating two core practical questions: (1) How can FINS protocol support be properly implemented within an open-source HMI platform such as FUXA to achieve reliability and performance parity with its natively supported protocols? (2) How can stable, low-latency read and write operations over FINS be ensured to enable accurate real-time visualization without packet loss or data staleness? To this end, a dedicated FINS communication module was designed and integrated into FUXA. There were two main goals: (1) Integrate FINS functionality with dependability and feature completeness on par with current built-in drivers (such as Modbus and OPC UA); (2) Empirically verify consistent data exchange and real-time UI updates in a physical deployment. The resulting solution facilitates digital transformation initiatives in manufacturing environments that heavily rely on Omron automation technology by offering a flexible, affordable, and vendor-independent HMI alternative.

2. Related Works

Open-source HMI/SCADA systems have gained increasing attention as alternatives to proprietary industrial automation platforms, particularly due to their flexibility, extensibility, and lower deployment cost. Several studies have explored the design and implementation of open-source HMI/SCADA architectures across different industrial domains.

Uddin et al. [11] developed an open-source SCADA system for a solar-powered reverse osmosis plant using Node-RED, Grafana, and InfluxDB. Their work shows that low-cost and community-driven SCADA architectures can be effectively implemented using open-source technologies. Similarly, Omidi et al. [12] proposed a modular Node-RED-based SCADA system for hybrid renewable energy generation, illustrating the suitability of lightweight web technologies for real-time monitoring with minimal computational resources.

In the power system domain, Almas et al. [13] integrated open-source SCADA with Phasor Measurement Units (PMUs) using the DNP3 protocol. Their results highlight protocol constraints—particularly polling limitations—that have implications for SCADA performance in large-scale grid environments.

In the field of industrial cybersecurity, Salazar et al. [14] introduced ICSNet, a hybrid-interaction honeynet incorporating multiple industrial protocols such as Modbus TCP, IEC-104, and Ethernet/IP. Notably, their implementation uses the FUXA HMI as the visualization layer, demonstrating FUXA's applicability beyond traditional HMI use cases.

Other studies have focused on web-based SCADA systems in smaller-scale environments. Hazdi et al. [15] implemented a web-based SCADA system for home automation using PLCs and OPC, demonstrating the feasibility of web technologies for real-time domestic control. Additionally, Thushara [16] introduced the FUXA web-based SCADA tool and discussed its deployment on edge devices using Modbus and MQTT, reinforcing its role as a modern and extensible HMI/SCADA platform.

Beyond these monitoring-oriented implementations, several works have examined deeper protocol integration strategies. Gil et al. [17] analyzed the convergence of multiple IoT and automation protocols in industrial electrical systems, highlighting the challenges of data interoperability across heterogeneous communication layers. Their study demonstrates that effective protocol integration requires not only transport-level compatibility but also harmonized data models, structured message encoding, and deterministic synchronization—elements often missing in lightweight or ad hoc SCADA implementations. Their findings emphasize that protocol integration is not merely a matter of adding communication drivers, but requires systematic treatment of interoperability, semantics, and data governance.

In contrast to the descriptive implementations of open-source HMI/SCADA systems in the aforementioned studies, which primarily rely on built-in support for widely adopted protocols such as Modbus, OPC UA, and MQTT, Giehl et al. [18] demonstrated a more advanced protocol-integration approach by

extending the OpenPLC framework with OPC UA server functionality through the open62541 library. This method ensures IEC 62541 compliance, enables security features such as message signing and encryption, and maintains compatibility with real-world OT components including HMIs and historians.

Similarly, our work integrates the proprietary FINS protocol into FUXA by developing a custom `DeviceFins` driver using the `omron-fins` Node.js library. Unlike OPC UA–based approaches that rely on information modeling and secure communication stacks, the FINS driver focuses on UDP/TCP socket handling, memory-area operations (e.g., DM, CIO), and efficient event emission for real-time UI updates. When compared with the protocol integration principles discussed in Gil et al. [17], our implementation addresses interoperability at the transport and memory-access layer rather than at the semantic or model-driven layer, filling a practical but previously unaddressed need in Omron-centric manufacturing systems.

Across these works, open-source SCADA implementations consistently rely on widely supported protocols such as Modbus, OPC UA, or MQTT. However, none of the reviewed studies address the Factory Interface Network Service (FINS) protocol, which remains widely used in Omron PLC-based industrial environments. This absence indicates a significant research gap in open-source HMI/SCADA systems that support FINS communication.

Therefore, the present study contributes by developing and integrating a FINS communication module into the FUXA HMI platform. This fills an identified gap in open-source industrial automation research and enables direct interoperability with Omron PLCs, which are prevalent in manufacturing systems such as those at PT XYZ.

3. Methodology

3.1. Research Overview

This work uses a hands-on, iterative development approach to expand the open-source HMI platform FUXA with native support for the Omron FINS protocol. The project progressed through the usual phases—requirements analysis, system design, implementation, and evaluation stages—to make sure the new FINS module performs on par with the platform's existing drivers, particularly Modbus TCP. The overall development workflow is shown in Figure 1.

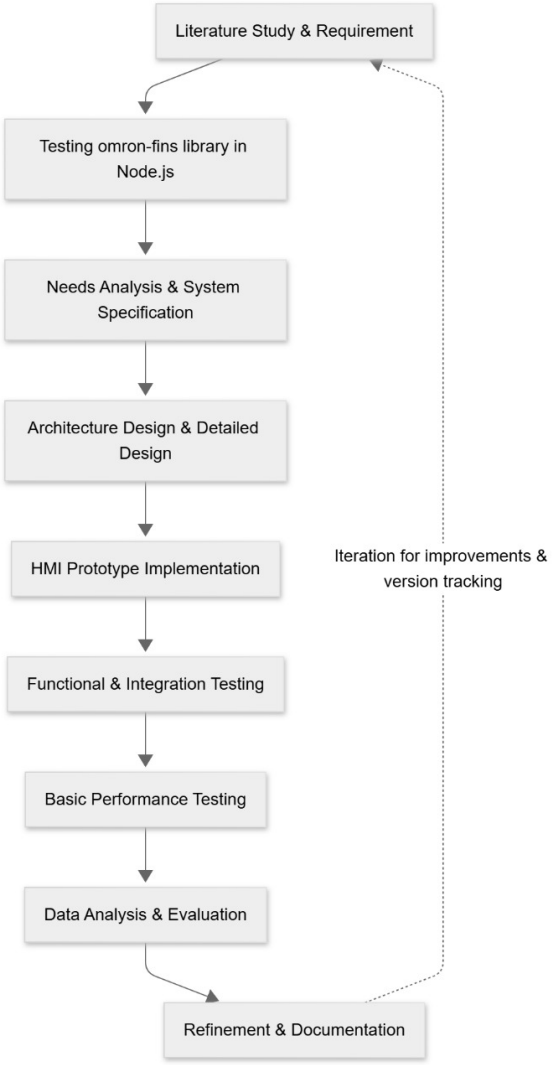

Figure 1: *Research workflow for FINS protocol implementation in FUXA*

3.2. System Architecture

The resulting system is built around three core layers, which are: (1) the physical PLCs, (2) the Node.js-based backend that handles all communication, and (3) the Angular frontend responsible for the actual HMI visualization. The Omron FINS support is added as a new device driver called `DeviceFins`, running entirely in FUXA's backend. This driver takes care of the low-level socket communication (UDP or TCP) and performs read/write operations on typical Omron memory areas such as `DM`, `CIO`, and `W`.

Data acquisition works by having the backend periodically poll the PLC—or react to specific triggers—pulling values from the configured memory addresses. These values are then propagated to

the frontend through the system's existing event emitter (`device-value:changed`) to update the HMI interface in real time. A simplified overview of this architecture is shown in Figure 2.

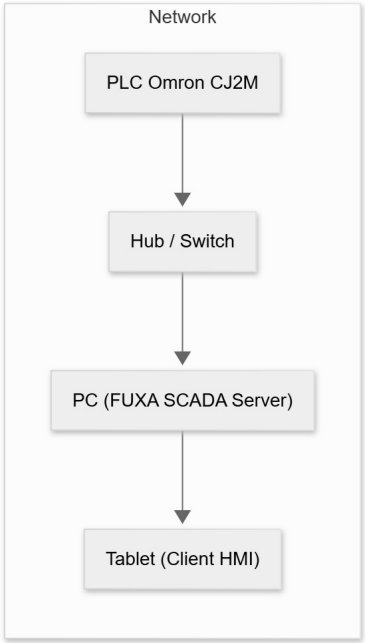

Figure 2: *Architecture of the developed HMI–PLC communication system*

3.3. Development Tools and Environment

The implementation and testing were conducted using this environment:

- **Programming Languages:** JavaScript (Node.js v18 LTS).
- **HMI Platform:** FUXA v1.2.13 (open-source web-based SCADA system).
- **PLC Device:** Omron CJ2M CPU34.
- **Operating System:** Windows 11.
- **Network Analysis Tools:** Wireshark.

3.4. Implementation Procedure

The development process was carried out in the following key phases:

1. **Protocol analysis and requirements analysis:** A detailed examination of the Omron FINS protocol was conducted, focusing on command structure, memory address read and write methods, and communication flow using the official Omron FINS Command Reference Manual.
2. **Design and implementation of the device driver:** The `DeviceFins`, a dedicated driver class, was developed within the existing FUXA backend architecture. This class encapsulates connection lifecycle management (TCP/UDP), read and write operations to standard memory areas, automatic reconnection logic with exponential back-off, timeout handling, and emission of value-change events.
3. **Frontend Integration:** The administrative web interface of FUXA was enhanced by introducing device-specific configuration fields—including PLC last three digits IP address (DA1), PC node address (SA1), IP address, port, and network mode—ensuring seamless integration with the platform's existing device management workflow.
4. **Testing and Debugging:** The implementation was rigorously tested through a combination of unit tests and real-world deployment on physical CJ2M CPU34 to confirm functional correctness, stability, and performance under various network conditions.

This structured, iterative approach ensured that the new FINS driver achieved full functional equivalence with the platform's established communication modules.

3.5. Testing Method

To assess the performance of the implemented FINS driver, a dedicated load-testing script was developed in Node.js. This script issued 1,000 successive read requests targeting memory area `D100` on the Omron PLC, while maintaining a concurrency level of ten simultaneous connections. The key metrics recorded throughout the experiment were:

- Average read/write latency (ms)
- Success rate of communication (%)
- CPU and memory usage
- Stability of reconnection handling during simulated disconnections

The results of these measurements were analyzed statistically to evaluate whether the implemented connector meets the performance characteristics comparable to Modbus TCP communication.

3.6. Evaluation Metrics

To ensure practical suitability for real-time HMI–PLC communication, the system was evaluated using five

key performance metrics. A compact summary of the metrics and their KPI thresholds is presented in Table 1.

Table 1: *Evaluation Metrics and KPI Targets*

Metric	KPI Target
Read/write latency	≤ 200 ms
Success rate	≥ 95%
CPU / Memory usage	≤ 30% / ≤ 500 MB
Alarm detection time	≤ 200 ms
Startup time	≤ 10 s

4. Results and Discussion

4.1. Evaluation of the `omron-fins` Library

The study began with a stress test of the `omron-fins` Node.js library to confirm the capability of FINS communication (read/write) between a PC and an Omron CJ2M PLC via UDP. The experimental setup is summarized in Table 2.

Table 2: *`omron-fins` trial environment configuration*

Component	Specification
PLC	Omron CJ2M-CPU34
Host Client	PC(i7-11th Gen, 16GB RAM, Win 11)
IP PLC	192.168.0.1
IP Client	192.168.0.234
Protocol	FINS/UDP (Port 9600)
Library	`omron-fins` (Node.js)
Runtime	Node.js v18 LTS

The Omron FINS library was installed via `npm` and validated using a stress test script that throws 1 000 read requests to address `D100` with a concurrency level of 10. The resulting network traces then captured using Wireshark to confirm frame structure and response codes.

Key observations from the library tests are:

- Captured responses consistently contained `Command Code 0101` (Memory Area Read) and `Response Code 0000` (no error), with incrementing Service IDs indicating unique matching responses.
- The library handled read operations reliably under the configured stress: average response latency was observed below 50 ms and the library exhibited correct framing behavior according to Omron FINS specification.
- No installation or dependency errors were encountered during setup; the library is suitable as a communication basis for further integration into the HMI system.

4.2. System Configuration and Integration

The FINS connector was integrated into the FUXA server under the directory `server/runtime/devices/fins`. The deployed testbed topology comprises one Omron CJ2M PLC, one PC acting as HMI server, and a tablet as the client (see Fig. 3). Typical connector parameters are listed in Table 3.

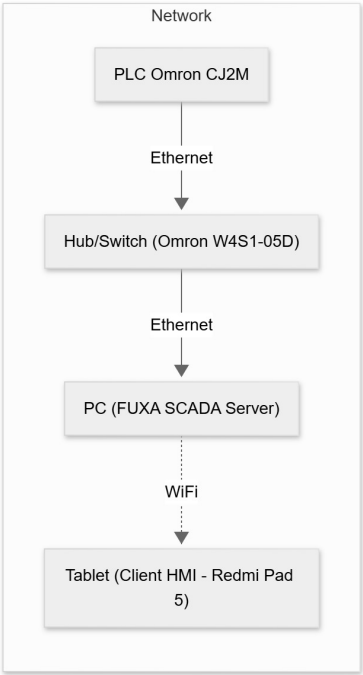

Figure 3: *System topology of the FUXA HMI with the FINS connector*

Parameter	Nilai Contoh
Alamat IP PLC	192.168.11.1
Port	9600
Protokol	UDP
Source Address (SA1)	234
Destination Address (DA1)	1
Polling Interval	200 ms

Table 3: *FINS connection configuration parameters*

After connector installation, initial verification was performed by issuing a single read to memory address

`D100`. Successful read/response without timeout confirmed that the connector was operational.

4.3. Implemented Functionality

The implemented FINS connector provides the following capabilities:

1. Transport-level connectivity (UDP and TCP) using the `omron-fins` library.

2. Periodic polling per device with the connector only reporting `connect-ok` after a successful read cycle.

3. Write (set) operations initiated from the UI and executed on the PLC in real time.

4. Event emission to FUXA runtime (`device-value:changed`, `device-status:changed`, `device-error`).

5. Frontend integration: device configuration form and tag editing dialog for FINS-specific parameters (SA1, DA1, memory area, address, data type).

Representative UI screens is shown in Figs. 4.

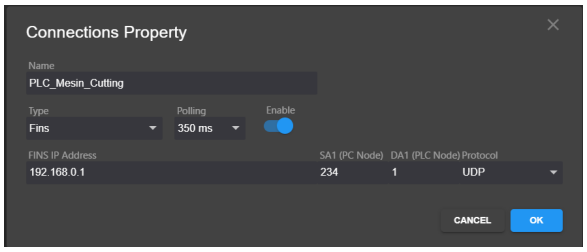

Figure 4: *UI Result — Device Settings: configuration form for FINS device (IP, port, SA1, DA1, protocol).*

After integrating the FINS connector module into both the backend and the HMI interface, a realtime communication test was conducted to verify bidirectional data transfer between the HMI system and the Omron PLC. Fig. 5 presents the HMI interface during the execution of the test.

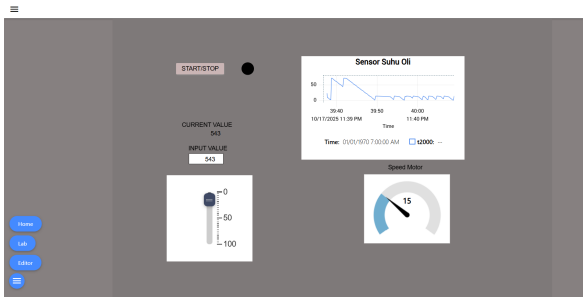

Figure 5: *HMI demonstration connected to the PLC in realtime.*

The HMI display includes several key elements:

- **Realtime trend (Oil Temperature Sensor):** Visualizes temperature values read from the PLC via the FINS protocol in real-time.
- **Slider and input field:** allow writing updated values to PLC memory addresses and change value in a specified memory via slider.
- **Motor speed gauge:** represents PLC variables using an analog-style visualization, demonstrating responsive data updates.

Experimental results show that PLC-to-HMI data updates occur within <100 ms, indicating that the implemented FINS connector provides low-latency and stable realtime communication.

In addition to realtime monitoring and control, the system featured an alarm mechanism to detect abnormal industrial process conditions that can make safety issues or warning. Fig. 6 shows an alarm triggered when the oil temperature exceeds the maximum threshold defined in the PLC.

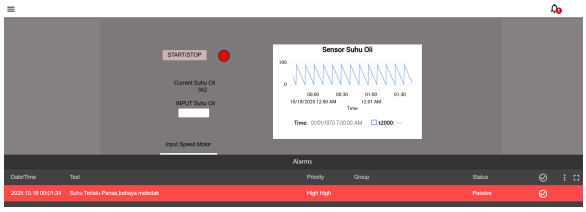

Figure 6: *Realtime alarm display when oil temperature exceeds the safety threshold.*

The alarm subsystem operate in realtime with the following characteristic:

- **Continuous monitoring:** PLC tag value are periodically sampled and compared with predefined threshold.
- **Alarm visualization:** triggered alarm appear in red with *High-High* priority to highlight critical condition.

- **Status synchronization:** alarm state (active, acknowledged, cleared) automatically update based on backend data.
- **FINS-based event detection:** each alarm directly correspond to PLC tags transmitted through the FINS UDP protocol, ensure fast and accurate detection.

Testing confirms that the system detect high-temperature events and displays alarms with a delay of less than 200,ms after the PLC value change. These results shows that the developed FINS connector reliably support both data exchange and realtime event notification within the HMI system.

4.4. Performance Evaluation

Performance evaluation comprised automated technical tests and a small-scale usability assessment. Results are summarized in Table 4.

Table 4: *Summary of Technical Evaluation Results*

Metric	Target	Observed
Read/write latency (avg.)	≤200 ms	12 ms
Communication success rate	≥95%	99%
Server CPU usage	≤30%	2%
Server memory usage	≤500 MB	462 MB
Alarm detection time	≤200 ms	1000 ms (formal)
Application startup time	≤10 s	3.14 s

4.4.1. Read/Write Latency

Read/write latency was measured by issuing a write to memory (e.g., `W1`) and immediately performing a read-back on the same address. The average latency across 100 repeated trial were **12,ms**, well below the 200,ms target.This result show that the integrated connector and FUXA frontend can achieve low round-trip times in the laboratory network condition used.

4.4.2. Communication Success Rate

The automated stress test program stresstest.js recorded **990 successful responses** and **10 failures** (99% success). Failures were predominantly timeouts visible in the Wireshark traces as requests without corresponding reply. Likely causes include UDP packet loss on the link, temporary PLC processing backlog, or aggressive scheduling from the test host. No systematic FINS framing errors were observed in successful captures.

4.4.3. Resource Utilization

System monitoring during load tests showed low CPU utilization (approx. 2–3%) and moderate memory consumption (backend ≈91 MB, frontend ≈371 MB). These figures indicate the Node.js + Angular architecture can operate efficiently on the selected PC hardware for the tested workload.

4.4.4. Alarm Detection Time

The alarm detection time is measured from the moment the triggering condition in the PLC become true until the alarm indicator appear on the HMI interface. The observed detection time is approximately 1000,ms, which is significantly slower than the KPI target of 200,ms. This delay happen because the minimum polling interval used by the HMI system to evaluate alarm conditions and generate alarm events is 1,s, as illustrated in Fig. 7.

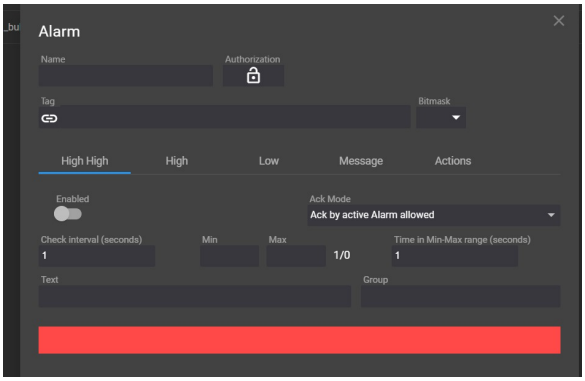

Figure 7: *HMI system alarm detection interval limitation.*

4.4.5. Application Startup Time

Measured application startup time averaged **3.14 s**, which satisfies the ≤10 s requirement and indicates acceptable initialization performance for the integrated system.

4.5. Usability Evaluation

A small usability survey (10 respondent: operators and technician) using a 5-point Likert scale assess configuration ease, clarity of status display, perceived

responsiveness, UI consistency, and overall satisfaction. Average score (scale 1–5) were:

- Configuration ease: 4.2
- Clarity of status: 4.1
- Responsiveness: 4.3
- UI consistency: 4.0
- Overall satisfaction: 4.3

Qualitative feedback highlighted appreciation for clear connection-status indicators and requests for a more prominent *Check Connection* control and a simplified logging mode for non-technical troubleshooting.

4.6. Discussion

The integration of a FINS connector into FUXA using the `omron-fins` library produce a functional, low-latency HMI capable of reliable read/write operation with Omron CJ2M PLCs under laboratory condition. Important takeaway points are:

- **Reliability:** Communication success rate (99
- **Performance:** Read/write latencies (12,ms) and low resource usage indicate the approach is suitable for responsive HMI operation on modest hardware.
- **Alarm timeliness:** The primary area needing improvement is alarm detection latency due to the current polling-based approach. Mitigation strategy include polling prioritization, reducing polling intervals carefully, or exploring event-driven mechanism if supported by the PLC network stack.
- **Robustness:** Observed timeout under high-stress scenario suggest adding retry backoff, packet pacing, and enhanced logging for root-cause analysis in noisy network.
- **Usability:** End-user feedback is positive; additional UX improvement and operator-focused logging would increase practical adoption.

5. Conclusion

This research successful implemented and evaluated the integration of the Factory Interface Network Service (FINS) protocol into the open-source Human–Machine Interface (HMI) system FUXA. The development introduce a new backend connector module (`DeviceFins`) written in Node.js and an extended frontend configuration interface in Angular, enable direct and seamless communication between FUXA and Omron PLCs. The connector support both UDP and TCP modes, allow read, write, and polling operation to be executed without the need for proprietary middleware or vendor-locked HMI system. As a result, FUXA become one of the first open-source HMI systems to natively support the Omron FINS protocol.

Based on experimental test, the implemented system show reliable and stable performance. The communication success rate reach 99%, and the average read/write latency was measured at 12 ms, well below the 200 ms KPI target. System resource consumption stay efficient, with average CPU usage of 2% and memory consumption of 462 MB during load testing. The HMI startup time average 3.14 s, indicate a responsive system suitable for small- to medium-scale production environment. However, the alarm detection delay reach approximately 1000 ms due to the internal one-second polling interval of the FUXA system. This limitation show that the alarm subsystem still rely on a time-driven polling approach rather than a fully event-driven mechanism.

Overall, the integration of FINS into FUXA successful fulfilled the two primary research objective: (1) enable the HMI system to communicate with Omron PLCs using the FINS protocol with functionality equivalent to existing Modbus support, and (2) improve the stability and visualization of data exchange on the HMI interface. Future research should focus on optimize the alarm handling mechanism through event-driven or interrupt-based approach, enhance network security with TLS or VPN, and expand multi-protocol interoperability. Additionally, open publication of the connector's source code, testing script, and dataset is encouraged to promote reproducibility and community-driven advancement in open-source industrial automation system.

References

[1] Mohsin Ali Koondhar et al. "The Role of PLC in Automation, Industry, and Education: A Review". In: *Pakistan Journal of Engineering Technology and Science* 11.2 (2023), pp. 22–31. DOI: 10.22555/pjets.v11i2.975.

[2] S. Mujawar. "Introduction to Human–Machine Interface". In: *Handbook/Book on Human–Machine Interfaces*. Chapter 1; Accessed via Wiley Online Library. Wiley, 2023. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781394200344.ch1 (visited on 09/26/2025).

[3] Aliyu Enemosah and Joseph Chukwunweike. "Next-Generation SCADA Architectures for Enhanced Field Automation and Real-Time Remote Control in Oil and Gas Fields". In: vol. 11. 12. International Journal of Computer Applications Technology and Research, 2022, pp. 514–529. DOI: 10.7753/IJCATR1112.1018. URL: https://doi.org/10.7753/IJCATR1112.1018.

[4] Lev Malyi and Matvei Bryksin. "Unified UI Design System for Industrial HMI Software Development". In: *Human-Computer Interaction. HCII 2024*. Vol. 14684. Cham: Springer, 2024, pp. 109–124. DOI: 10.1007/978-3-031-60405-8_8. URL: https://doi.org/10.1007/978-3-031-60405-8_8.

[5] A. Mohamed et al. "Off-The-Shelf IoT Gateways as a Viable Alternative to Traditional HMI Devices". In: *International Petroleum Technology Conference*. IPTC. Feb. 2024, D021S083R006. DOI: 10.2523/IPTC-24344-MS. eprint: https://onepetro.org/IPTCONF/proceedings-pdf/24IPTC/24IPTC/D021S083R006/3361298/iptc-24344-ms.pdf. URL: https://doi.org/10.2523/IPTC-24344-MS.

[6] McKinsey & Company. "Is industrial automation headed for a tipping point?" In: *McKinsey & Company Insights* (2023). URL: https://www.mckinsey.com/capabilities/operations/our-insights/is-industrial-automation-headed-for-a-tipping-point.

[7] Lalie Arnoud et al. "HENDRICS: A Hardware-in-the-Loop Testbed for Enhanced Intrusion Detection, Response and Recovery of Industrial Control Systems". In: *30th European Symposium on Research in Computer Security (ESORICS 2025), Workshop on Assessment with New methodologies, Unified Benchmarks, and environments, of Intrusion detection and response Systems (ANUBIS)*. Toulouse, France, Sept. 2025. URL: https://uca.hal.science/hal-05325191.

[8] Daniel Jakob et al. "A Hardware-in-the-Loop System for Monitoring Building Energy Consumption with Sparse Sensors". en. Feb. 2025. DOI: 10.13140/RG.2.2.13085.63208. URL: https://publica.fraunhofer.de/handle/publica/487277.

[9] "FrangoTeam — Home Page". https://frangoteam.org/. Accessed: 2025-12-06. 2025.

[10] Jianlei Gao et al. "A new Detection Approach against attack/intrusion in Measurement and Control System with Fins protocol". In: *2020 Chinese Automation Congress (CAC)*. 2020, pp. 3691–3696. DOI: 10.1109/CAC51589.2020.9327136.

[11] Sheikh Usman Uddin, Mirza Jabbar Aziz Baig, and Mohammad Tariq Iqbal. "Design and Implementation of an Open-Source SCADA System for a Community Solar-Powered Reverse Osmosis System". In: *Sensors* 22.24 (2022), p. 9631. DOI: 10.3390/s22249631. URL: https://www.mdpi.com/1424-8220/22/24/9631.

[12] Mohammad Omidi, Majid Salehifar, and Amir Mohammadi. "Design and Implementation of an Open Source SCADA System for Monitoring a Hybrid Renewable Power System". In: *Energies* 16.5 (2023), p. 2229. DOI: 10.3390/en16052229. URL: https://www.mdpi.com/1996-1073/16/5/2229.

[13] M. S. Almas and L. Vanfretti. "Open Source SCADA Implementation and PMU Integration". In: *IEEE PES General Meeting* (2014).

[14] Luis Salazar et al. "ICSNet: A Hybrid-Interaction Honeynet for Industrial Control Systems". In: *Proceedings of the Sixth Workshop on CPS&IoT Security and Privacy (CPSIoTSec'24)*. Salt Lake City, UT, USA: ACM, 2024, pp. 1–12. ISBN: 979-8-4007-1244-9. DOI: 10.1145/3690134.3694813. URL: https://doi.org/10.1145/3690134.3694813.

[15] Robby Hazdi, Muhammad Fadli, and Yusril Maulana. "Perancangan SCADA pada Sistem Otomatisasi Rumah". In: *eProceedings of Applied Science* 3.2 (2017). Universitas Telkom, pp. 1099–1106. URL: https://openlibrarypublications.telkomuniversity.ac.id/index.php/appliedscience/article/view/4046.

[16] Kasun Thushara. "Getting Start with FUXA - Web Based SCADA Tool". Accessed: 2025-07-16. 2024. URL: https://wiki.seeedstudio.com/reTerminal-DM_intro_FUXA/.

[17] S. Gil, G. D. Zapata-Madrigal, R. García-Sierra, et al. “Converging IoT protocols for the data integration of automation systems in the electrical industry”. In: *Journal of Electrical Systems and Information Technology* 9.1 (Feb. 10, 2022), p. 1. DOI: 10.1186/s43067-022-00043-4. URL: https://doi.org/10.1186/s43067-022-00043-4.

[18] Alexander Giehl, Michael P. Heinl, and Victor Embacher. “EmuFlex: A Flexible OT Testbed for Security Experiments with OPC UA”. In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*. ARES ’24. Vienna, Austria: ACM, July 2024, pp. 1–9. ISBN: 979-8-4007-1718-5/24/07. DOI: 10.1145/3664476.3670931. URL: https://doi.org/10.1145/3664476.3670931.